# Interpretable vs. Explainable Algorithms

## Key Definitions

- Interpretable model — a model whose reasoning process is transparent by design (e.g., decision trees, rule lists, scoring systems, GAMs). The model's own structure is the explanation; no extra step is needed.
- Explainable model — a black-box model (e.g., deep neural network, large ensemble) whose behaviour is approximated after training by a separate, simpler explainer (e.g., LIME, SHAP, saliency maps).

> ***Example:*** *A single decision tree that predicts loan approval directly from income and credit score is interpretable. A deep neural network that predicts loan approval, paired afterwards with a LIME explanation of that prediction, is explainable.*

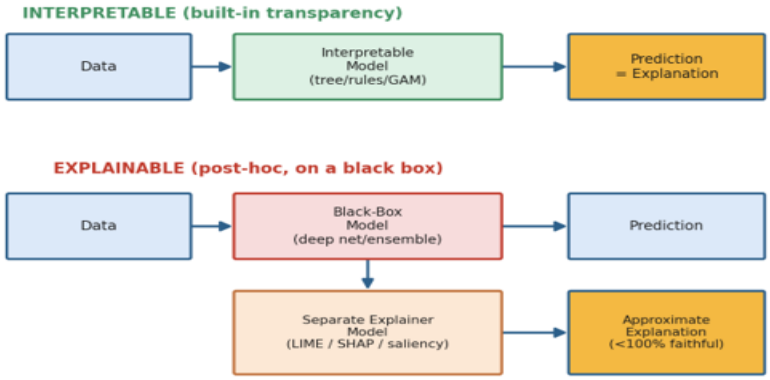



*Figure A: Interpretable models are transparent by design; explainable models bolt a second, approximate model onto a black box.*

# Issues With Post-Hoc Explanations on Black-Box Models

five main issues with relying on post-hoc explanation methods instead of interpretable models:

### The accuracy–interpretability trade-off is a myth:

- Empirical comparisons across many domains show interpretable models can match black-box accuracy — the assumed trade-off often does not hold.
- Using an interpretable model lets practitioners see and fix data/model issues directly, which fits naturally into iterative processes such as KDD or CRISP-DM.

> ***Example:*** *In several credit-scoring and recidivism studies, simple scoring systems performed nearly as well as complex black-box classifiers.*

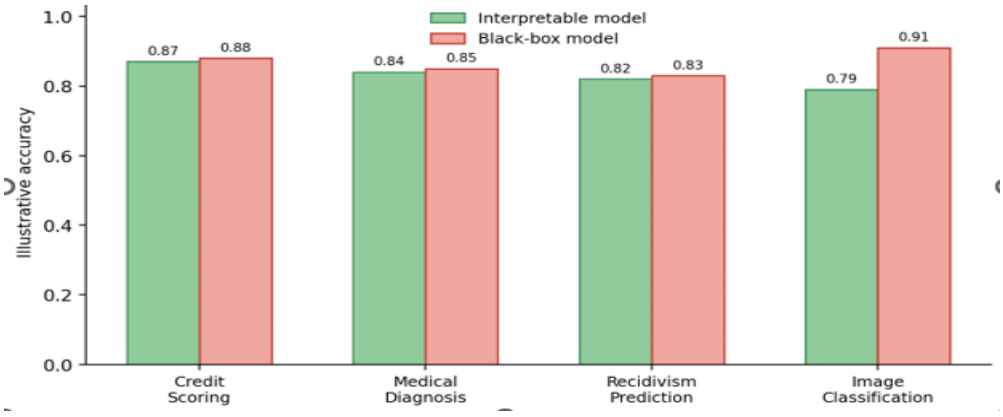



*Figure B: The accuracy gap between interpretable and black-box models is often small (illustrative values).*

## Post-hoc explanations are not 100% faithful:

- An explanation method is itself a second, simplified model approximating the black box — not the black box's true reasoning.
- If an explainer is only, say, 90% faithful, roughly 10% of its explanations may misrepresent what the real model is doing — risky in high-stakes settings.
- Relying on explanations means trusting two logical models instead of one: the original black box and the explainer.

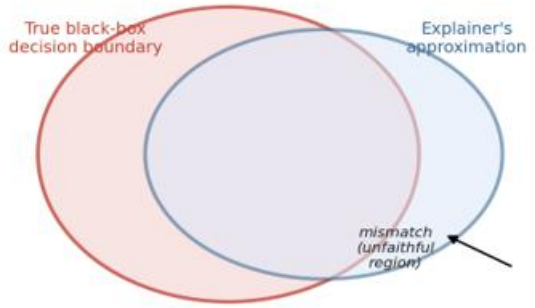



*Figure C: The explainer's approximated boundary does not perfectly match the true black-box boundary — the mismatch is where explanations mislead.*

## Explanations often lack sufficient detail:

- Explanation techniques may show what the model attended to, without showing why or how that led to the prediction.
- Example: saliency maps in computer vision highlight which pixels the classifier focused on, but give no information about the model's actual decision logic.

> ***Example:*** *A saliency map might highlight a dog's ear in an image classified as 'dog', but it cannot tell us why the model weighted that ear so heavily, or whether it is using it correctly.*

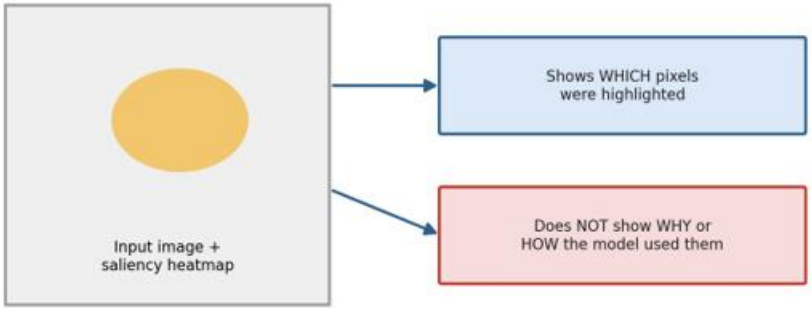



*Figure D: Saliency maps show which pixels mattered ("what") but not the model's reasoning ("why").*

## Relevant factors can lie outside the modelled data:

- Some decisions depend on context that isn't captured in the training data at all.
- Example: in criminal justice, the circumstances surrounding a crime may matter for bail or parole decisions, but a model trained only on crime-record data cannot see them.
- Without an interpretable, transparent model, it is hard to know when such external factors should override the model's output.

## Explanations can create complex decision pathways:

- Some post-hoc methods generate elaborate rule chains or decision paths to explain a black box's prediction.
- The added complexity of the explanation itself increases the chance of human error when someone tries to interpret it.

For high-stakes decisions — credit, bail and parole, medical diagnosis —building genuinely interpretable models from the start, rather than relying on post-hoc explanations of black boxes. Interpretability removes the need to trust a second, approximate model and makes it easier to catch and fix problems in the data or model directly.

# Ensemble-Based Techniques for Interpretability:

## Ensemble Learning Foundations:

Before looking at interpretable ensemble techniques, it helps to review the three basic strategies used to build ensembles: Bagging, Boosting, and Stacking. Each combines multiple models differently, which affects both accuracy and interpretability.

### Bagging (Bootstrap Aggregating):

- Multiple models are trained in PARALLEL, each on a random bootstrap sample (sampled with replacement) of the training data.
- All models are typically the same type (e.g., decision trees).
- Final prediction = average of outputs (regression) or majority vote (classification).
- Main benefit: reduces variance and overfitting. Example algorithm: Random Forest.

> ***Example:*** *To predict whether an email is spam, we train 5 decision trees, each on a different bootstrap sample of the email dataset. If 3 out of 5 trees say 'spam', the final prediction is spam (majority vote).*

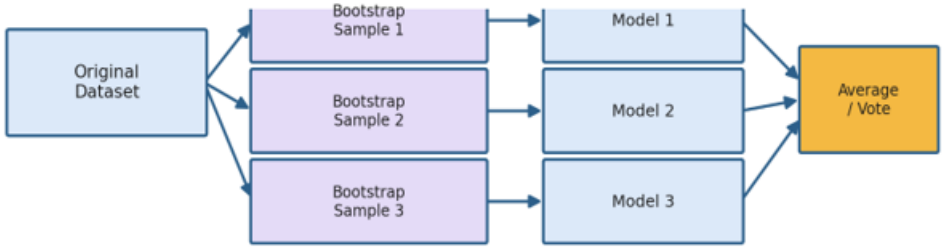



*Figure E: Bagging — parallel models trained on bootstrap samples, combined by averaging/voting.*

### Boosting:

- Multiple models are trained SEQUENTIALLY, where each new model focuses on correcting the errors of the previous ones.
- Misclassified or poorly-predicted examples are given more weight/attention in the next round.
- Final prediction = weighted sum/vote of all models.
- Main benefit: reduces bias and builds a strong learner from weak learners. Example algorithms: AdaBoost, Gradient Boosting, XGBoost.

> ***Example:*** *To predict loan default, Model 1 (a shallow tree) misclassifies a few high-income defaulters. Model 2 is trained giving those specific cases more weight, so it learns to correct that mistake. Model 3 then corrects whatever Model 1 and 2 still get wrong.*

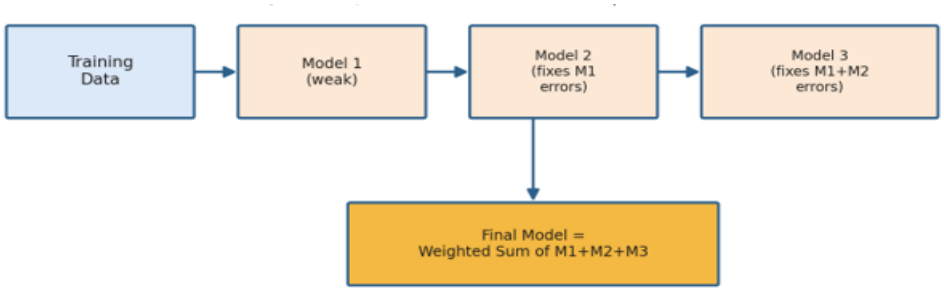


*Figure f: Boosting — sequential models where each one fixes the previous model's errors.*

## Stacking (Stacked Generalization)

- Different types of base models (e.g., KNN, Decision Tree, SVM) are trained in parallel on the same data.
- A separate meta-model is then trained to learn how to best combine the predictions of the base models.
- Final prediction comes from the meta-model, not a simple vote or average.
- Main benefit: can combine the strengths of very different model types.

> ***Example:*** *To predict customer churn, we train a KNN, a Decision Tree, and an SVM. Their three predictions are then fed as inputs into a Logistic Regression meta-model, which learns the best way to weigh each base model's opinion to produce the final churn prediction.*

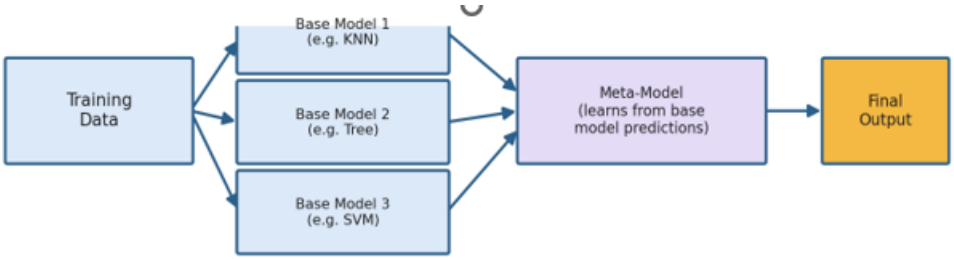



*Figure g: Stacking — diverse base models combined by a learned meta-model.*

Ensemble methods such as Random Forests and Gradient Boosting typically trade off interpretability for accuracy, since they combine many trees whose joint decision logic is hard for a human to follow. To address this, several techniques have been developed that make ensembles interpretable — either by extracting simple rules from them, or by constraining their structure so that each component remains human-understandable. Five key approaches are discussed below: Boosted Rulesets, Skope-Rules, RuleFit, Explainable Boosting Machines (EBM), and Iterative Random Forest (iRF).

# 1. Boosted Rulesets

- Builds an ensemble of simple IF–THEN rules using boosting.
- Each new rule is added to correct the errors made by the previous rules (sequential, additive learning).
- Final prediction is a weighted vote / weighted sum of all rule outputs.
- More interpretable than boosted decision trees, because each individual rule can be read and understood on its own.

> ***Example:*** *In credit-risk scoring: Rule 1 — IF income < 20,000 AND late_payments > 3 THEN high risk. Rule 2 — IF credit_history > 5 years THEN low risk.*

***Interpretability Metrics — Boosted Rulesets***

| **Properties** | **Values** |
|---|---|
| Local or global | Global (with local rule-level explanations) |
| Linear or non-linear | Non-linear (axis-parallel rules) |
| Monotonic or non-monotonic | Non-monotonic |
| Feature interactions captured | Yes (within each rule) |
| Model complexity | Medium |

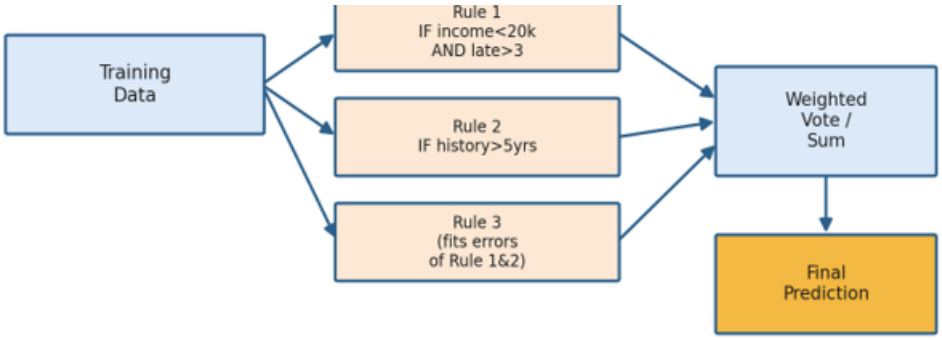



*Figure 1: Boosted Rulesets — rules are learned sequentially and combined by a weighted vote.*

# 2. Skope-Rules

- A rule-extraction technique that combines bagging of decision trees with rule selection.
- Rules are selected based on precision and recall thresholds — only the most reliable rules are kept.
- Extracts only the most useful, human-readable rules from an ensemble of trees, discarding weak or redundant ones.

> ***Example:*** *In fraud detection, Skope-Rules might output: IF transaction_amount > 10,000 AND location = foreign THEN flag as fraud (precision ≈ 92%).*

***Interpretability Metrics — Skope-Rules***

| **Properties** | **Values** |
|---|---|
| Local or global | Global and local |
| Linear or non-linear | Non-linear (axis-parallel rules) |
| Monotonic or non-monotonic | Non-monotonic |
| Feature interactions captured | Yes |
| Model complexity | Low to medium |

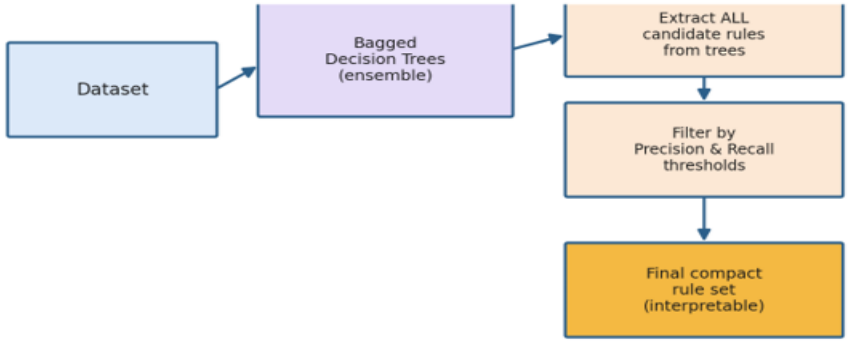



*Figure 2: Skope-Rules — candidate rules are extracted from a bagged tree ensemble and filtered by precision/recall.*

# 3. RuleFit

- Combines decision rules extracted from a tree ensemble (Random Forest / GBM) with linear regression.
- Extracted rules are treated as new binary features (1 if the rule fires, 0 otherwise).

- A sparse linear model (e.g., Lasso) is then trained on the original features plus the extracted rule-features.
- Result: a model where each rule and each original feature has its own interpretable coefficient / contribution.

> ***Example:*** *In predicting house prices: Rule — IF rooms > 4 AND location = urban, learned coefficient ≈ +$50,000 on the predicted price.*

***Interpretability Metrics — RuleFit***

| Properties | Values |
|---|---|
| Local or global | Global and local |
| Linear or non-linear | Non-linear (can be axis parallel) |
| Monotonic or non-monotonic | Non-monotonic |
| Feature interactions captured | Yes |
| Model complexity | Medium to high |

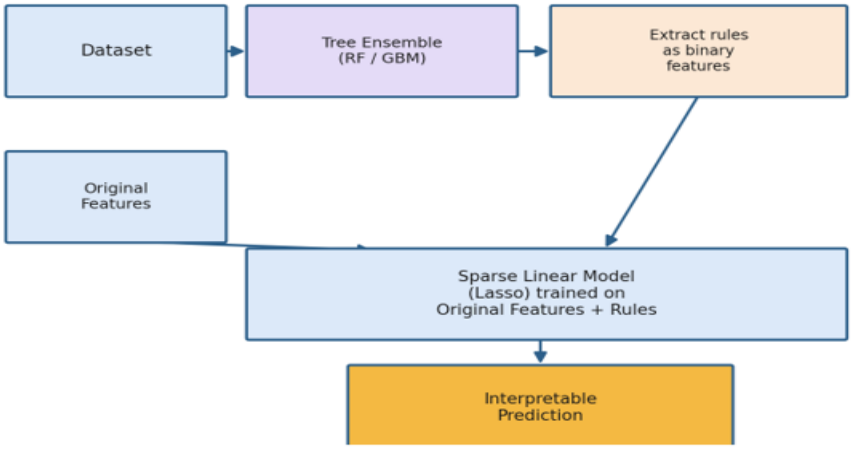



*Figure 3: RuleFit — rules extracted from a tree ensemble become features in a sparse linear model.*

# 4. Explainable Boosting Machines (EBM)

- A Generalized Additive Model (GAM) variant, trained using gradient boosting.
- Learns one shape function per feature (and optionally pairwise interaction terms).
- Because the model is additive — prediction = f1(x1) + f2(x2) + ... — each feature's contribution can be plotted and read independently.
- Achieves accuracy close to black-box ensemble models while remaining fully transparent.

> ***Example:*** *In diabetes-risk prediction, an EBM shows a graph of how risk changes with BMI, independent of how it changes with Age — each curve is a separate, readable shape function.*

***Interpretability Metrics — Explainable Boosting Machine (EBM)***

| Properties | Values |
|---|---|
| Local or global | Global and local |
| Linear or non-linear | Non-linear per feature (globally additive/linear in shape functions) |

| Properties | Values |
|---|---|
| Monotonic or non-monotonic | Non-monotonic (monotonicity can be enforced as a constraint) |
| Feature interactions captured | Yes (pairwise interaction terms only) |
| Model complexity | Medium |

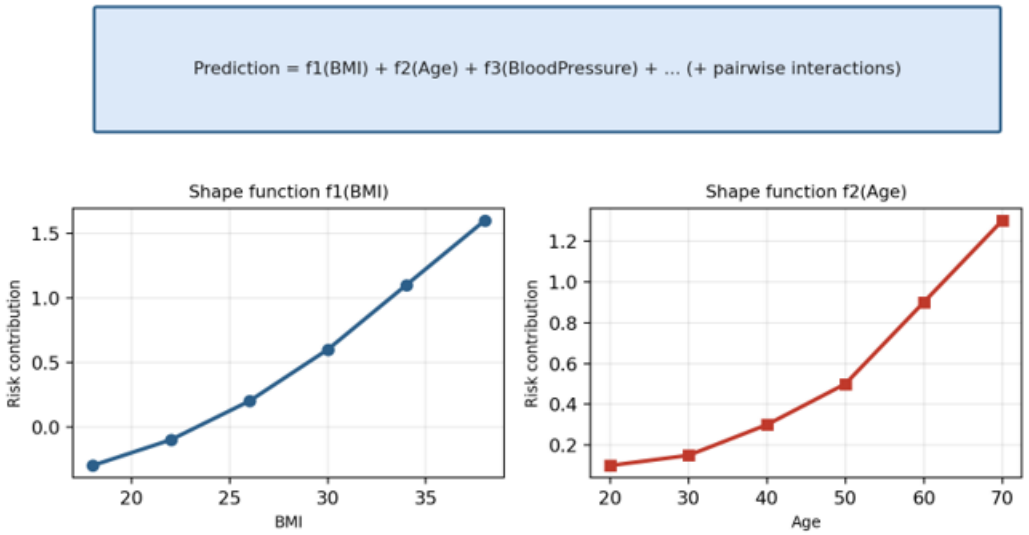
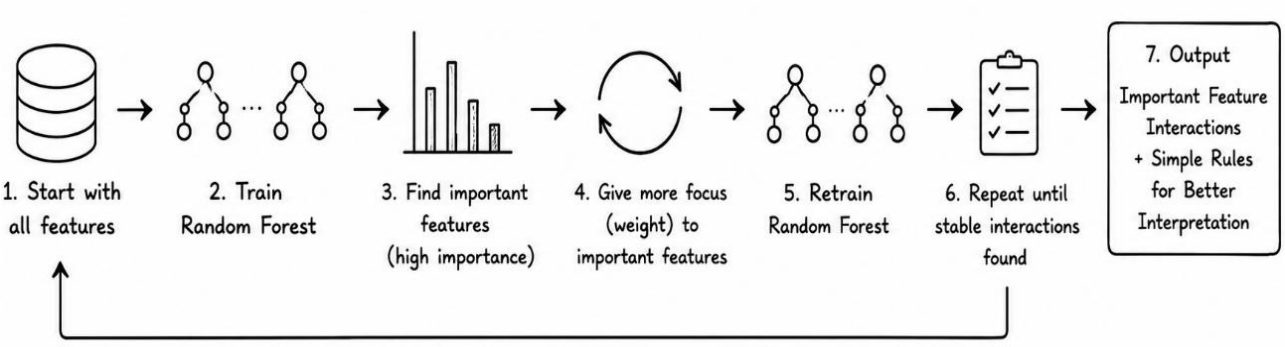



*Figure 4: EBM — the additive structure lets each feature's shape function be plotted and interpreted separately.*

# 5. Iterative Random Forest (iRF)

- Extends Random Forests by iteratively reweighting features based on their importance from the previous iteration.
- Across iterations, the forest is refit repeatedly, gradually emphasising features that matter most.
- Focuses on detecting stable, high-order feature interactions across many trees, not just ranking individual feature importances.

> ***Example:*** *In genomics, iRF has been used to identify interacting sets of genes (e.g., gene1 AND gene5 AND gene12 together) that jointly predict a biological outcome, rather than ranking genes individually.*




***Interpretability Metrics — Iterative Random Forest (iRF)***

| Properties | Values |
|---|---|
| Local or global | Global |

| Properties | Values |
|---|---|
| Linear or non-linear | Non-linear |
| Monotonic or non-monotonic | Non-monotonic |
| Feature interactions captured | Yes (specialises in high-order interactions) |
| Model complexity | High |

These ensemble-based interpretability techniques bridge the gap between the high accuracy of ensemble models and the need for transparency. Boosted Rulesets, Skope-Rules, and RuleFit extract explicit rules; EBM enforces an additive structure so each feature can be inspected on its own; and iRF uncovers stable, interpretable feature interactions hidden inside a Random Forest.